
Software Implementation Report

for

Autonomous Vehicle Simulation and Vulnerability Analyzer

Version 1.0

Prepared by:

Raider Security

Project 14

Date: 10/16/2025

Webpage: <https://gingerbreadgames.github.io/CapstoneWebPage/>

Video Demo: <https://www.youtube.com/watch?v=5d4voGpssJg>

Reid Layne
Tyler Bowen
Gage Johnson
Nick Sanchez

Section 001
Section 001
Section 001
Section 001

reilayne@ttu.edu
tylbowen@ttu.edu
gage.d.johnson@ttu.edu
nicolas232.sanchez@ttu.edu

Table of Contents

Project Overview	2
Design Details	3
UML Class Diagram	4
Use Case Diagram	7
Sequence Diagram	10
Implementation Details	15
System Architecture	15
Technology Stack:	16
File Structure:	16
Vulnerability Injection Tab:	16
Analysis Tab:	16
Report Generation Tab:	16
Attack Implementations	17
1. CMD_VEL Topic Injection	17
2. Odometry Sensor Spoofing	17
3. IMU Data Spoofing	17
4. Parameter Server Manipulation	18
5. Node Shutdown via XMLRPC	18
Custom Vulnerability Injection	18
1. Topic Discovery	19
2. Attack Templates	19
Technical Challenges	20
Group Log	22

Project Overview

Project Description

The Autonomous Vehicle Simulation and Vulnerability Analyzer is a comprehensive security research platform designed to demonstrate, analyze, and document vulnerabilities in ROS (Robot Operating System)-based autonomous vehicles. Developed in partnership with the Army Research Lab, this tool provides researchers and security

professionals with a controlled environment to study attack vectors, validate threat models, and test defensive mechanisms. The system operates on the Clearpath Husky UGV platform in simulation, offering five distinct vulnerability demonstrations that exploit fundamental security gaps in ROS's architecture: lack of authentication, absence of encryption, unrestricted topic access, unprotected XMLRPC endpoints, and world-writable parameter server.

Motivation

Military and commercial autonomous systems increasingly rely on ROS for robot control and coordination. However, ROS was designed for trusted research environments and lacks built-in security features. This creates critical vulnerabilities in deployed systems that could be exploited by adversaries. The Army Research Lab identified the need for a practical tool that could:

- **Demonstrate Real Threats:** Show concrete examples of how ROS vulnerabilities can be exploited
- **Validate Research:** Provide empirical evidence supporting academic threat models
- **Train Personnel:** Educate developers and operators about ROS security risks
- **Test Defenses:** Create a baseline for evaluating security solutions like ROS-Defender

- **Generate Data:** Produce reproducible attack scenarios for research publications

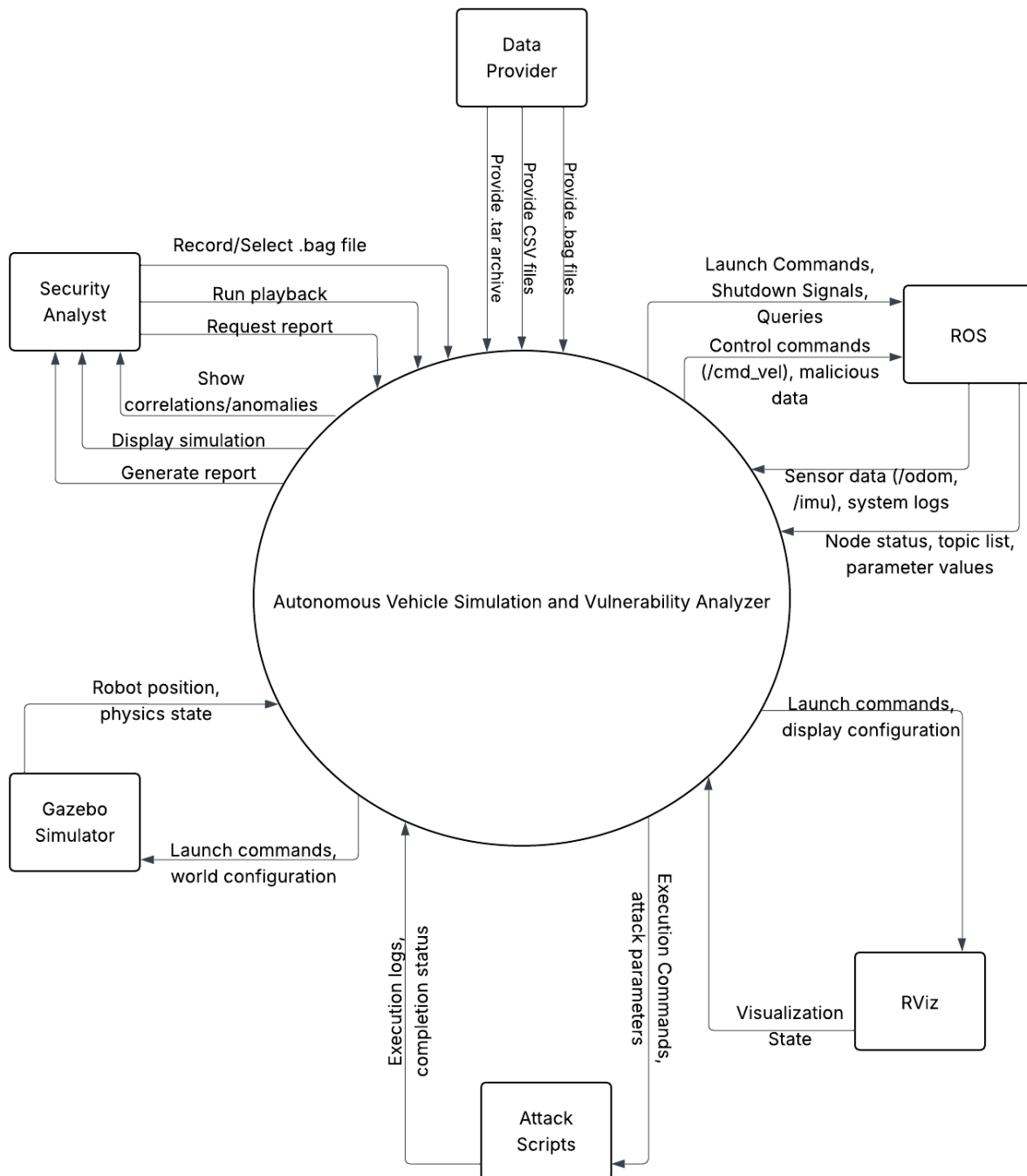
AVSVA addresses these needs by combining simulation control, attack injection, data recording, and analysis capabilities into a single, user-friendly application.

Key Features

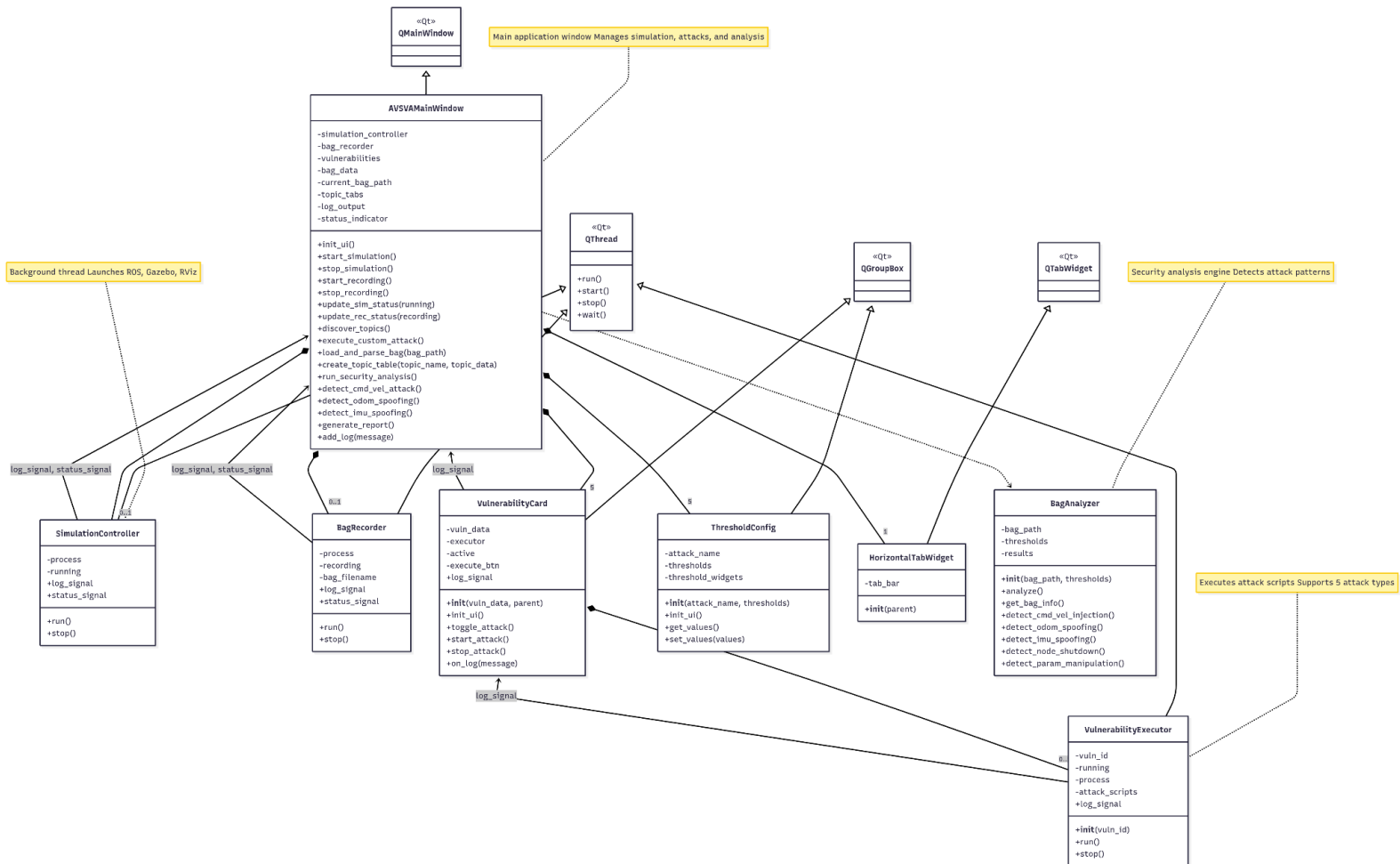
- **Simulation Playback:** Load and replay ROS bag files to observe vehicle behavior
- **Data Characterization:** Analyze CSV sensor data for correlations and statistical anomalies
- **Vulnerability Testing:** Inject controlled anomalies to test system resilience
- **Interactive Visualization:** Generate plots, heatmaps, and trajectory displays
- **Comprehensive Reporting:** Export analysis results in text and PDF formats

Design Details

Context Diagram



UML Class Diagram



AVSVAMainWindow**Inherits from:** QMainWindow (PyQt5)**Role:** Central application controller and primary user interface

- Manages the entire lifecycle of simulation, recording, attack execution, and security analysis
- Coordinates all background threads and GUI components
- Displays real-time logs, status indicators, topic data tables, and attack results
- Implements core functionality including simulation control, bag recording, vulnerability execution, and post-attack analysis
- Key methods: start_simulation(), stop_simulation(), start_recording(), execute_custom_attack(), run_security_analysis(), add_log(), update_sim_status()

SimulationController**Inherits from:** QThread**Role:** Background thread responsible for launching and managing the full ROS + Gazebo + RViz simulation

- Executes launch_husky_auto_drive.sh to start the Husky robot in a simulated environment
- Monitors process state and emits log_signal and status_signal for GUI updates
- Ensures clean termination using process groups

BagRecorder**Inherits from:** QThread**Role:** Records ROS bag files during attack scenarios

- Automatically generates timestamped bag files

(husky_attack_YYYYMMDD_HHMMSS.bag)

- Records critical topics including /cmd_vel, /odom, /imu/data, /tf, etc.
- Provides real-time recording status feedback

VulnerabilityExecutor**Inherits from:** QThread**Role:** Executes one of five supported attack scripts in a separate process

- Maps vulnerability IDs to attack scripts (e.g., cmd_vel_injection → attack_cmd_vel.py)

- Supports five attack types:

1. CMD_VEL Injection
2. Odometry Spoofing
3. Node Shutdown
4. Parameter Manipulation
5. IMU Spoofing

- Forwards attack output via log_signal

VulnerabilityCard**Inherits from:** QGroupBox**Role:** Visual widget representing a single attack type in the GUI

- One card per attack (5 total)
- Contains attack name, description, Execute/Stop button, and live status
- Owns and controls a VulnerabilityExecutor instance
- Provides immediate visual feedback during attack execution

ThresholdConfig**Inherits from:** QGroupBox**Role:** Configurable panel for tuning attack detection sensitivity

- Allows users to adjust detection thresholds per attack type

- Values are passed to BagAnalyzer during security analysis

HorizontalTabWidget

Inherits from: QTabWidget

Role: Custom tab widget with horizontal orientation

- Improves usability when displaying data from many ROS topics
- Dynamically populated with topic tables during bag analysis

BagAnalyzer

Role: Offline security analysis engine

- Loads recorded attack bag files
- Implements detection algorithms for all five attack types
- Detection methods include:
 - detect_cmd_vel_injection() – conflicting velocity commands
 - detect_odom_spoofing() – impossible position jumps
 - detect_imu_spoofing() – unrealistic sensor readings

- detect_node_shutdown() – sudden topic disappearance

- detect_param_manipulation() – unexpected parameter changes

- Generates detailed analysis report displayed in the GUI

Key Design Strengths

- **Thread Safety:** All long-running operations (simulation, recording, attacks) run in dedicated QThreads to keep the GUI responsive
- **Modularity:** New attacks can be added by creating a script and a 'VulnerabilityCard'
- **Real-World Fidelity:** Directly models the actual implemented and tested system
- **Signal/Slot Architecture:** Clean communication between threads and GUI using PyQt5 signals
- **Separation of Concerns:** Clear responsibilities between control, execution, analysis, and presentation

Use Case Diagram

The use cases in this diagram represent modular behaviors that align with the system's decomposed subsystems (objects or modules), allowing for independent implementation while showing dependencies. Each primary use case corresponds to a core module:

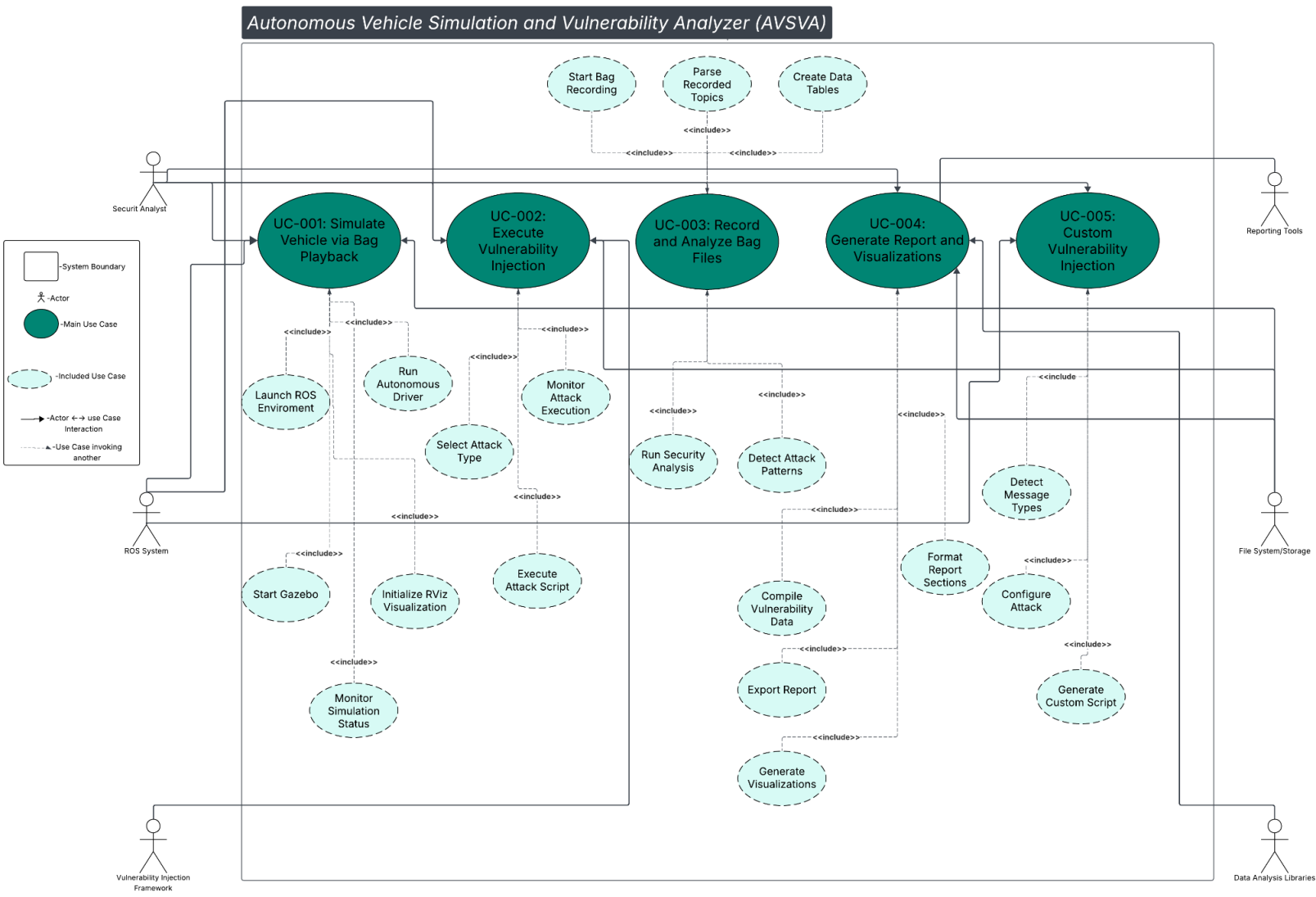
- **UC-001: Simulate Vehicle via Bag Playback** – This use case corresponds to the Simulation Module, which serves as an object responsible for launching and managing the entire ROS simulation environment. It includes sub-use cases like "Launch ROS Environment" (initializing roscore and ROS nodes), "Start Gazebo Simulation" (launching the 3D vehicle simulation), "Initialize RViz Visualization" (setting up sensor visualization), "Run Autonomous Driver" (executing navigation algorithms), and "Monitor Simulation Status" (tracking system health). The module interacts with external systems, such as the ROS System for node management and the Gazebo Simulator for visual feedback, utilizing the SimulationController QThread class to manage the simulation lifecycle asynchronously.
- **UC-002: Execute Vulnerability Injection** – Linked to the Vulnerability Injection Module, this use case represents an attack orchestration object that executes predefined security tests against the running simulation. It includes "Select Attack Type" (choosing from CMD_VEL injection, odometry spoofing, node shutdown, parameter manipulation, or IMU spoofing), "Execute Attack Script" (launching Python attack scripts as separate processes), and "Monitor Attack Execution" (tracking attack progress via VulnerabilityExecutor QThread). This module integrates with the Vulnerability Injection Framework for script execution and the ROS System for publishing malicious messages to topics, requiring UC-001 to be active before attacks can be launched.
- **UC-003: Record and Analyze Bag Files** – This corresponds to the Data Capture and Analysis Module, an object focused on recording attack scenarios and detecting security vulnerabilities. It includes "Start Bag Recording" (capturing ROS topics via rosbag during attacks), "Parse Recorded Topics" (extracting data from recorded files), "Create Data Tables" (formatting topic data with conditional highlighting), "Run Security Analysis" (executing vulnerability detection algorithms), and "Detect Attack Patterns" (identifying spin attacks, position jumps, and abnormal sensor readings). The module leverages the BagRecorder QThread for asynchronous recording, BagAnalyzer for parsing, and Data Analysis Libraries (pandas, numpy, scipy) for statistical pattern detection, monitoring UC-002 to capture attack behaviors in real-time.
- **UC-004: Generate Report and Visualizations** – Aligned with the Reporting Module, this use case models a comprehensive documentation object that compiles security findings. It

includes "Compile Vulnerability Data" (aggregating identified vulnerabilities with severity ratings), "Format Report Sections" (organizing executive summary, vulnerability details, attack logs, and mitigation strategies), "Generate Visualizations" (creating plots and charts via Data Analysis Libraries), and "Export Report" (producing PDF/HTML outputs through Reporting Tools). This module depends on results from UC-003 for vulnerability data and integrates with File System/Storage for report persistence, providing preview functionality before final export.

- **UC-005: Custom Vulnerability Injection** – This represents the Advanced Attack Configuration Module, an object enabling researchers to create novel attack scenarios beyond predefined templates. It includes "Detect Message Types" (identifying topic message structures), "Configure Attack Parameters" (setting custom payload values), and "Generate Custom Script" (creating Python attack code on-the-fly). This module interacts with the ROS

System for topic discovery, Vulnerability Injection Framework for script generation and execution, and the File System/Storage for saving reusable attack templates, requiring UC-001 to provide an active simulation environment.

Overall, the diagram's modules (use cases) promote a modular, thread-based design where each can be implemented as a QThread-based class or subsystem with clear interfaces for asynchronous operations. The actor (Security Analyst) interacts via the GUI (MainWindow), which orchestrates these use cases through signal-slot mechanisms. External systems, including ROS System, Gazebo Simulator, Vulnerability Injection Framework, File System/Storage, Data Analysis Libraries, and Reporting Tools, provide specialized functionality that the use cases leverage. Relationships using <<include>> ensure reusable sub-functions across modules, while dependencies between use cases establish clear execution order and data flow, facilitating extensibility and parallel development in the software design.

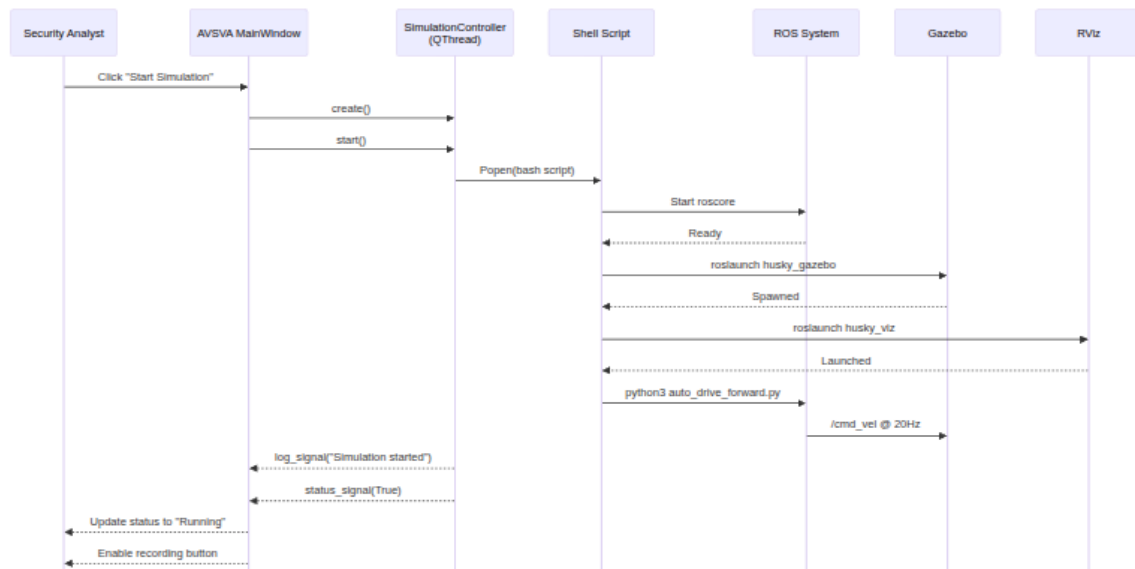


Sequence Diagram

1. Simulate Vehicle via Bag Playback

Allows the analyst to launch a complete ROS simulation environment with the Clearpath Husky robot. The system executes a shell script that starts roscore, launches

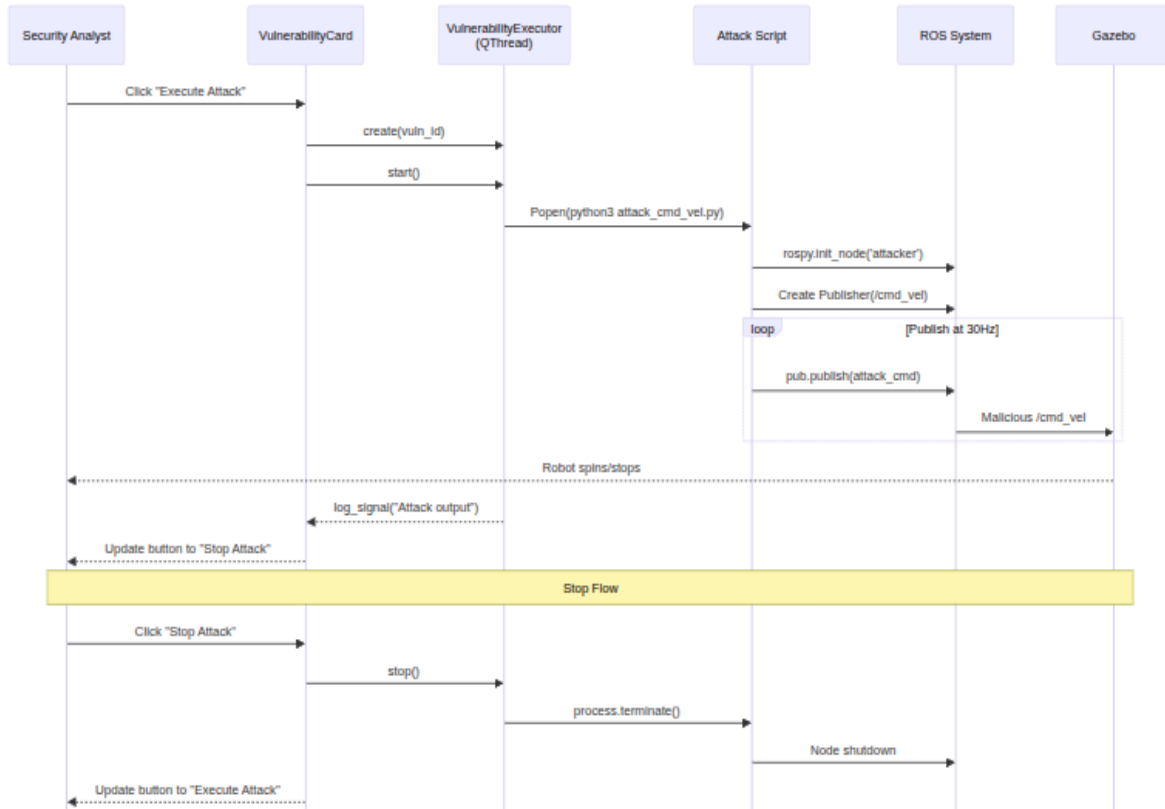
Gazebo with the robot model, opens RViz for sensor visualization, and runs the autonomous driving node. The SimulationController QThread manages the entire lifecycle and emits signals to update the GUI status.



2. Execute Vulnerability Injection

Facilitates security testing by executing predefined attack scripts against the running simulation. The system launches Python attack scripts (CMD_VEL injection, odometry spoofing, node shutdown,

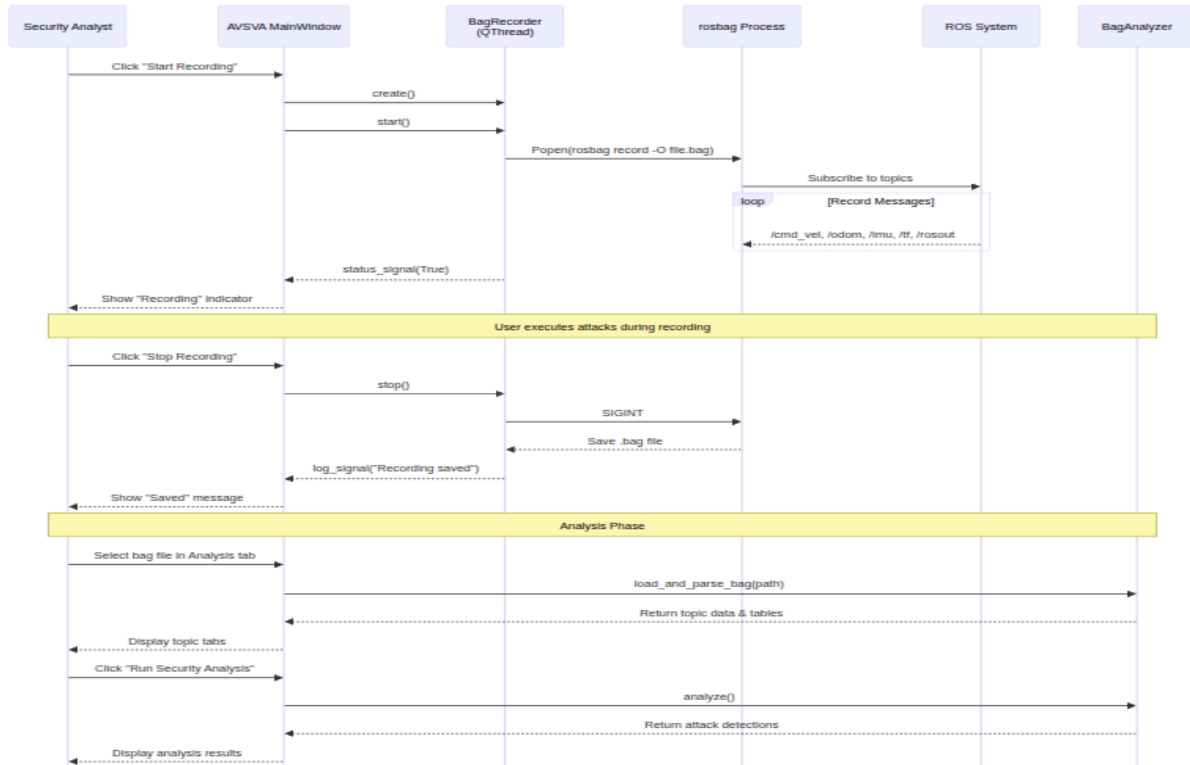
parameter manipulation, IMU spoofing) as separate processes that publish malicious messages to ROS topics. The VulnerabilityExecutor QThread monitors attack execution and provides real-time feedback through the Vulnerability



3. Record and Analyze Bag Files

Enables capture and analysis of ROS topic data during attack scenarios. The BagRecorder QThread executes rosbag to record critical topics (/cmd_vel, /odom, /imu/data, /tf, /rosout) with timestamped

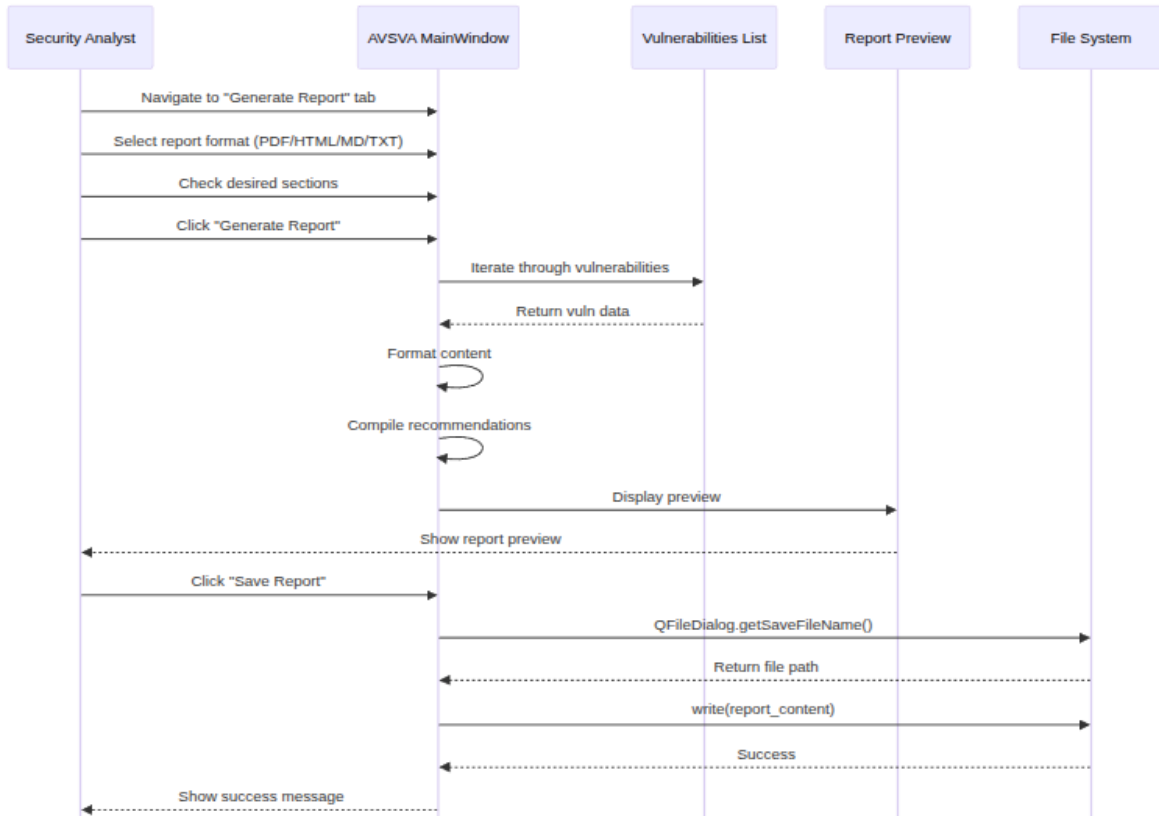
filenames. The BagAnalyzer parses recorded files, creates topic-specific data tables with conditional formatting, and runs security analysis algorithms to detect attack patterns such as spin attacks, position jumps, and abnormal sensor readings.



4. Generate Report and Visualizations

Compiles all security findings into comprehensive reports. The system iterates through identified vulnerabilities, formats including executive summary, vulnerability details, attack logs, and mitigation strategies.

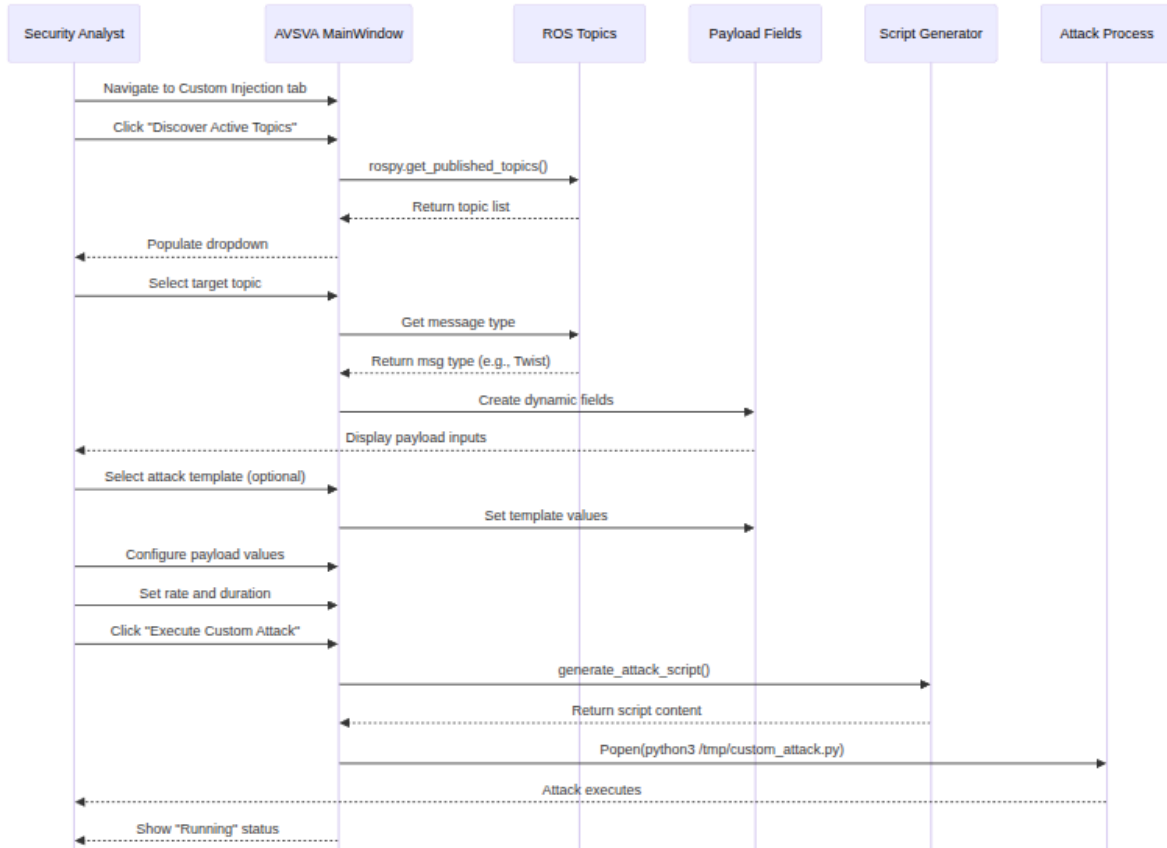
detailed descriptions with severity ratings, compiles security recommendations, and displays a preview. Reports can be exported) with customizable sections



5. Custom Vulnerability Injection

Allows researchers to create custom attack parameters for advanced security testing. The system discovers active ROS topics, detects message types, and dynamically

generates payload input fields. Users can select attack templates or configure custom values, and the system generates Python attack scripts on-the-fly that are executed as separate processes against the simulation.

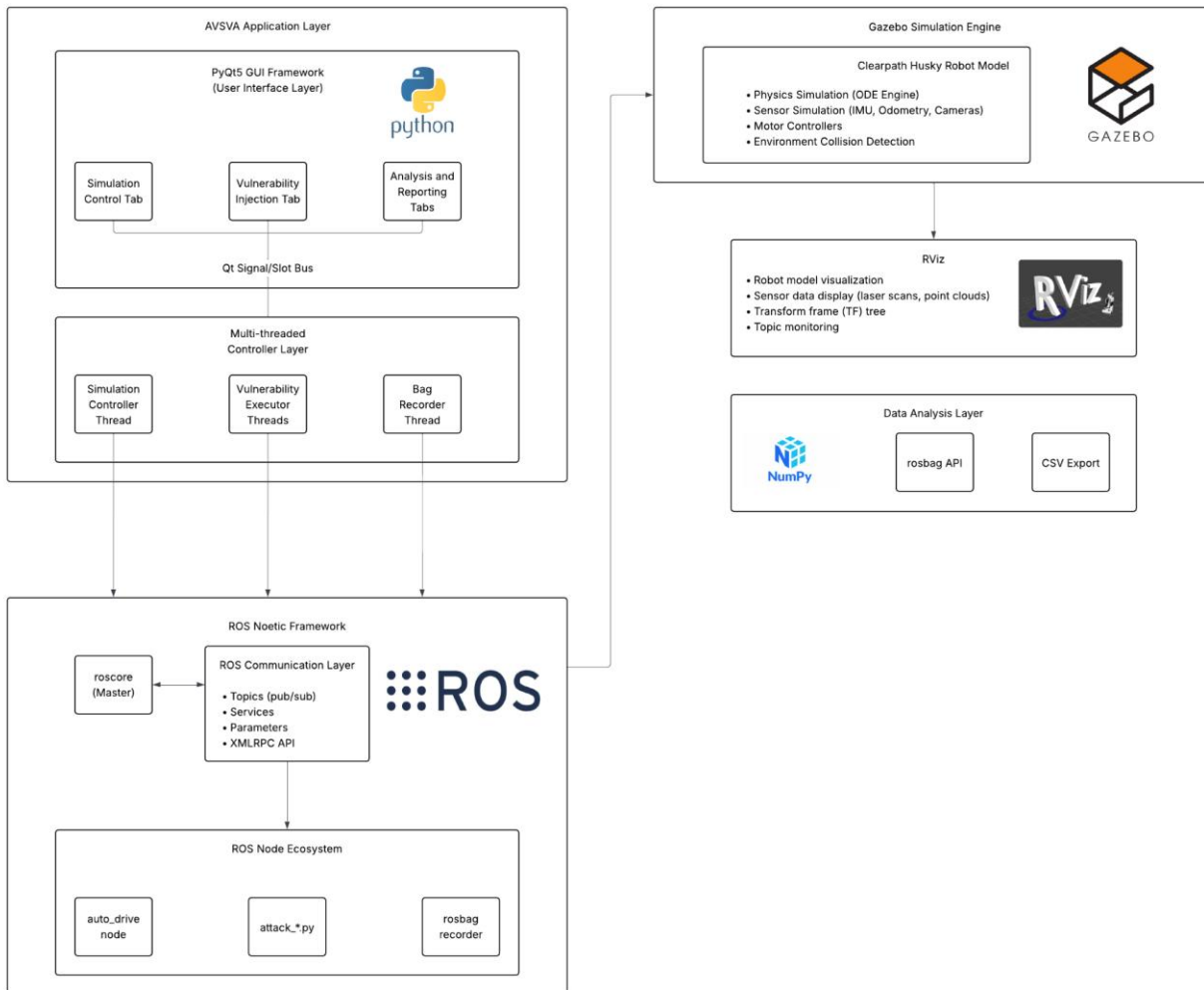


Implementation Details

System Architecture

AVSVA is implemented as a standalone desktop application built with Python and PyQt5. The system integrates with ROS 1 Noetic for robot simulation and attack execution, using Gazebo for 3D visualization. Unlike the original design

proposal which envisioned separate modules for data analysis and trajectory simulation, the final implementation focuses on vulnerability demonstration and forensic analysis as a single monolithic application.



Technology Stack:

Core Language: Python 3.8+ with PyQt5 for GUI

Robot Framework: ROS 1 Noetic for robot control and communication

File Structure:

The project consists of three main components:

avsva_app.py: Main monolithic application containing all GUI, simulation control, attack management, bag analysis, and reporting functionality

attack_scripts: Five standalone Python scripts demonstrating ROS vulnerabilities

System Architecture

AVSVAMainWindow - Main GUI class with four-tab interface (Simulation, Vulnerability Injection, Analysis, Report)

SimulationController (QThread) - Manages Gazebo/RViz in background thread

BagRecorder (QThread) - Records ROS topics during attacks

VulnerabilityExecutor (QThread) - Executes attack scripts as separate processes

VulnerabilityCard - Widget displaying attack info with execute button

BagAnalyzer - Forensic analysis engine with five detection algorithms

ThresholdConfig - Configurable detection parameters

Robot Simulation Tab:

- Start/stop controls for Gazebo simulation
- Bag file recording during attacks
- Real-time system log with timestamps

Vulnerability Injection Tab:

- Preset mode: Five pre-configured attacks with educational content

Simulation: Gazebo simulator with Clearpath Husky UGV model

Data Recording: rosbag for recording, NumPy for numerical analysis

Visualization: PyQt5 widgets with conditional formatting, RViz for robot state

simulation_scripts: Gazebo launch and autonomous driving scripts

- launch_husky_auto_drive.sh - Bash script launching Gazebo, RViz, and autonomous navigation
- auto_drive_forward.py - Simple forward driving node

- attack_imu.py - IMU data manipulation
- attack_param.py - Parameter server manipulation
- attack_shutdown.py - Node shutdown via XMLRPC
- attack_odom.py - Odometry sensor spoofing
- attack_cmd_vel.py - CMD_VEL topic injection attack
- Custom mode: Build custom attacks with topic discovery, templates, and script export

Analysis Tab:

- Load and parse bag files
- Per-topic data tables with conditional formatting (red=attack, yellow=warning, green=normal)
- Automated attack detection with confidence scores
- CSV export capability

Report Generation Tab:

- Configurable sections (summary, vulnerabilities, recommendation)

Attack Implementations

AVSVA demonstrates five distinct vulnerability classes through standalone Python attack scripts. Each script exploits fundamental security gaps in ROS's architecture and runs as an independent process that can be executed during live simulation.

1. CMD_VEL Topic Injection

Attack Mechanism: Publishes malicious velocity commands to `/husky_velocity_controller/cmd_vel` at 30Hz, overwhelming the legitimate autonomous driving controller (20Hz). The attack script sends conflicting commands with `linear.x=0.0` (stop) and `angular.z=2.0` rad/s (rapid spin), creating the signature "spin attack" pattern.

Exploited Vulnerability: ROS topics allow multiple publishers without authentication or arbitration. The system has no mechanism to verify message sources or establish command priority, meaning the last message received typically takes precedence regardless of origin.

Observable Impact: The Husky robot loses its intended forward trajectory and begins spinning in place. The navigation stack continues operating but receives conflicting velocity commands, demonstrating complete loss of motion control. This attack is immediately visible in both Gazebo simulation and bag file analysis through the characteristic zero linear velocity combined with high angular velocity signature.

2. Odometry Sensor Spoofing

Attack Mechanism: Publishes false odometry messages to `/husky_velocity_controller/odom` with fabricated position data (`x=-10.0, y=5.0`)

and unrealistic velocity values (`linear.x=-0.5` m/s backward motion). The fake odometry creates impossible position jumps that violate physical constraints while maintaining realistic message structure.

Exploited Vulnerability: ROS provides no cryptographic authentication or integrity verification for sensor data. Nodes consuming odometry messages cannot distinguish legitimate sensor readings from attacker-generated spoofed data, allowing adversaries to manipulate the robot's perceived position and velocity.

Observable Impact: The navigation and localization systems receive incorrect state information, causing the path planner to make decisions based on a false belief about the robot's position. Detection analysis reveals sudden position discontinuities exceeding 5 meters between consecutive messages and velocity values outside the physically realistic range (-1.0 to 10.0 m/s).

3. IMU Data Spoofing

Attack Mechanism: Injects false inertial measurement data to `/imu/data` with impossible sensor readings including angular velocities of 10 rad/s (approximately 573°/second) and linear accelerations of 50 m/s² (over 5g). The attack uses sinusoidal modulation to create persistent oscillating disturbances that confound sensor fusion algorithms.

Exploited Vulnerability: IMU messages lack source authentication and physical constraint validation. The system accepts sensor data at face value without verifying whether readings fall within mechanically possible ranges for the hardware, allowing injection of values that exceed physical sensor limitations.

Observable Impact: State estimation filters incorporating IMU data receive corrupted inertial measurements, degrading localization accuracy and potentially causing instability in control systems. The attack is detectable through statistical analysis showing sustained periods where angular velocity exceeds 5.0 rad/s or linear acceleration exceeds 20.0 m/s², with more than 10% of messages flagged as anomalous.

4. Parameter Server Manipulation

Attack Mechanism: Exploits the unprotected ROS parameter server to modify runtime configuration values. The attack executes a sequence of malicious parameter changes: first setting `/husky_auto_drive/linear_speed` to -5.0 m/s (causing unexpected reverse motion), then 100.0 m/s (dangerously exceeding hardware limits), and finally 0.0 m/s (forcing a stop).

Exploited Vulnerability: The ROS parameter server implements no access control lists (ACLs) or write permissions. Any node in the system can read or modify any parameter without authentication, and there is no audit trail of parameter modifications or validation of parameter value ranges.

Observable Impact: Nodes that periodically re-read parameters receive malicious configuration values, causing erratic behavior as speed settings change unpredictably. The autonomous driving node exhibits sudden velocity changes exceeding the 2.0 m/s threshold, with behavior oscillating between reverse motion, excessive speeds, and complete stops as parameters are manipulated.

Custom Vulnerability Injection

Beyond the five preset attacks, AVSVA provides a Custom Injection interface that

5. Node Shutdown via XMLRPC

Attack Mechanism: Uses ROS's XMLRPC administrative interface to remotely terminate the `/husky_auto_drive` node. The attack queries the ROS Master for the target node's URI, establishes a direct XMLRPC connection to the node, and invokes the `shutdown()` method with an arbitrary shutdown message.

Exploited Vulnerability: Every ROS node exposes an unauthenticated XMLRPC interface for administrative functions. The shutdown API requires no credentials or authorization, allowing any process with network access to terminate any node in the system. There is no distinction between legitimate administrative shutdowns and malicious attacks.

Observable Impact: The autonomous driving controller terminates immediately, causing complete loss of velocity command publication. The robot stops receiving motion commands, effectively achieving denial-of-service. Detection analysis identifies this attack through sudden topic silence, with message gaps exceeding the 2.0 second threshold indicating unexpected node termination.

Each attack script is instrumented with detailed logging (visible via `rospy.loginfo`) that reports attack parameters, target topics, and execution status. This logging is captured in bag files and displayed in the AVSVA GUI, providing researchers with complete visibility into attack execution and enabling forensic analysis of vulnerability exploitation patterns.

enables security researchers to develop novel attack scenarios against arbitrary ROS topics. This feature transforms AVSVA from a demonstration tool into a flexible

security testing platform for exploratory research and defense validation.

Capability Overview

The Custom Injection interface operates through a four-quadrant workflow:

1. Topic Discovery - Researchers can query the live ROS system to identify all active topics and their message types. This enables testing of proprietary topics, debug interfaces, or custom message formats not covered by preset attacks.

2. Attack Templates - Eight configurable templates provide starting points for common attack patterns: stop commands, spin attacks, reverse motion, maximum speed, position spoofing, sensor noise injection, value freezing, and fully custom payloads. Templates automatically populate payload fields based on the selected topic's message structure.

3. Dynamic Payload Configuration - The system introspects message types (Twist, Odometry, IMU, etc.) and generates appropriate input fields for each message component. Researchers specify exact values for fields like linear velocities, angular velocities, positions, and accelerations, enabling precise control over attack characteristics.

4. Execution Parameters - Configurable publish rate (1-100 Hz) and duration settings allow researchers to test rate-dependent vulnerabilities, timing attacks, and sustained versus burst attack patterns.

Research Applications

Custom injection supports several advanced security research scenarios:

- **Boundary Testing:** Inject extreme values to discover input validation gaps and identify where systems fail gracefully versus catastrophically
- **Chain Attacks:** Sequence multiple injection phases (e.g., IMU spoofing followed by odometry manipulation) to amplify attack effects
- **Defense Validation:** Test proposed security mechanisms like rate limiting, message authentication, or anomaly detection by crafting attacks designed to bypass specific defenses
- **Threshold Calibration:** Systematically vary attack intensity to tune detection parameters, balancing false positive and false negative rates
- **Zero-Day Discovery:** Explore attack vectors not documented in existing literature, particularly for custom ROS implementations or proprietary message types

Script Export and Reproducibility

The "Save as Python Script" feature exports custom attacks as standalone, self-contained Python scripts. This enables:

- **Peer Review:** Share attack implementations with the research community for validation
- **Automated Testing:** Integrate attacks into continuous security testing pipelines
- **Publication Support:** Generate proof-of-concept code for academic

papers demonstrating novel vulnerabilities

- **Cross-Platform Testing:** Export attacks developed on AVSVA for execution on physical robots or alternative simulation environments

Generated scripts include all researcher-configured parameters (topic names, payload values, publish rates) and can be executed independently of the AVSVA GUI, facilitating integration with external security tools and frameworks.

Integration with Detection Analysis

Custom attacks integrate seamlessly with AVSVA's configurable detection system. Researchers can:

1. Execute a custom attack against the simulation
2. Record the attack scenario in a bag file
3. Load the bag file in the Analysis tab
4. Adjust detection thresholds in the Detection Settings tab
5. Re-run security analysis to measure detection accuracy
6. Iterate until optimal detection parameters are identified

This closed-loop workflow enables researchers to develop both attacks and corresponding defenses within a single platform, accelerating the security research cycle from vulnerability discovery to mitigation deployment.

Technical Challenges

Challenge 1: Understanding RViz and Gazebo Integration

One of the initial hurdles was understanding how RViz and Gazebo work together in the ROS ecosystem. Both displayed 3D visualizations of the robot, creating confusion about their relationship. Through systematic study of ROS documentation and experimentation, we learned that Gazebo is a physics simulation engine that runs the robot model and publishes sensor data, while RViz is purely a visualization tool that displays data published to topics. Using tools like `roslaunch tf view_frames` helped clarify the transform tree structure and coordinate system relationships. Once this was clear, integrating both tools became straightforward, the simulation controller launches Gazebo for physics and RViz for visualization in sequence, with both subscribing to the same topics to provide realistic simulation and detailed sensor monitoring.

Challenge 2: Learning to Write Attack Scripts

Developing vulnerability demonstration scripts required learning ROS security research from scratch with no prior experience. We started by using command-line tools like `rostopic pub` to manually publish messages and observe effects, which provided immediate feedback. Each attack script was developed in isolation, beginning with simple `CMD_VEL` injection, progressing to odometry spoofing with quaternion math, and finally implementing XMLRPC and parameter attacks. Reading ROS API documentation, studying message definitions with `rosmmsg show`, and examining existing code patterns all contributed to learning.

Through this iterative process, five fully functional attack scripts were developed, each validated independently and capable of running standalone for testing.

Challenge 3: Maintaining UI Responsiveness

Early prototypes suffered from UI freeze issues where clicking "Start Simulation" would lock the application for 10-15 seconds while Gazebo loaded. The root cause was subprocess calls executing in the main GUI thread using blocking methods. The solution was redesigning with PyQt5's QThread framework, moving each major operation to dedicated worker threads. Communication uses Qt's signal-slot mechanism, worker threads emit signals with data, and the main thread processes them via callbacks. Qt automatically queues signals from background threads and handles them in the main thread's event loop, eliminating race conditions. This threading architecture ensures the application remains fully responsive during all operations while users can read logs, switch tabs, and cancel operations mid-stream.

Challenge 4: Clean Process Termination

Testing revealed that quitting AVSVA left zombie processes running—roscore, Gazebo, and RViz continued consuming resources, forcing users to manually execute killall commands. The issue stemmed from Python's

subprocess module not automatically killing child processes when the parent terminates. The solution used `os.setsid()` when spawning processes to create process groups, then `os.killpg()` to send signals to entire groups. The `closeEvent()` handler ensures cleanup runs even if users close via the X button. This approach provides reliable cleanup where all spawned processes terminate cleanly within seconds, with testing confirming zero zombie processes after termination.

Challenge 5: Bag File Analysis Performance

Initial bag file analysis suffered from severe performance problems where loading 500MB files took 2-3 minutes and froze the UI. The root cause was inefficient sequential file I/O and repeated parsing, each analysis operation re-read the entire file. The solution required redesigning to parse files once at load time, reading all messages into memory organized in nested dictionaries. NumPy arrays store message field values, enabling vectorized operations that provide 100-1000x speedup over Python loops. Progress indicators provide feedback during initial parsing. Performance improved dramatically, 500MB bags now parse in 5 seconds, and once loaded, all operations (filtering, searching, statistics) are near-instantaneous, making interactive exploration practical.

Group Log

Gage Johnson

gage.d.johnson@ttu.edu

Work done:

Built out the GUI, connected all parts of the program to work together using shell scripts and subprocessing. Lead development for the simulation through Gazebo, RViz, and .bag recording.

Contribution Percentage: 25%

Reid Layne

reilayne@ttu.edu

Work done:

Contributed to documentation and refinement of use case scenarios for the vulnerability analysis workflow, led research on RViz, loading the correct configuration of .bag files into RViz.

Contribution Percentage: 25%

Tyler Bowen

tylbowen@ttu.edu

Work done:

Revised class diagram & context diagram. Lead research on ROS1 noetic critical vulnerabilities to develop attack scripts and build out the injection part of the application.

Contribution Percentage: 25%

Nick Sanchez

nicolas232.sanchez@ttu.edu

Work done:

Revised sequence diagram, led development of the .bag file loading function and data parser. Contributed to the analysis design in the GUI and how dynamic detection thresholds should work.

Contribution Percentage: 25%